T. Sethu
192521470
CSA1713

1. Vacuum-cleaning Robot

a) PEAS Description

	Description
→ Performance	Clean both rooms, save time and energy.
→ Environment	Two rooms, each clean or dirty
→ Actuators	Left Right, Suck, Do nothing.
→ Sensors	Dirt sensor and location sensor.

b) Suitable Intelligent Agent

* If the room is dirty, it sucks the dirt
* If the other room needs cleaning it moves there
* If both rooms are clean it stops

2. Goal State: The destination point

3. Properties of Task Environment

Property	Chess	Self-driving car	Crossword
observability	Fully	Partially	Fully
Determination	Deterministic	Stochastic	Deterministic
Tasktype	Sequential	Sequential	Sequential
Environment	Static	Dynamic	Static
State type	Discrete	Continous	Discrete.

→ Chess and crossword puzzles have visible, discrete states. A self-driving car faces changing traffic and uncertainty, making its environment dynamic and partially observable

3. Formal Search Problem.

A delivery robot moves through a grid to reach its destination while avoiding obstacles.

1. Initial State: The robots starting position.

3. 2. Goal state: The delivery point

3. Actions: Move up, Down Left or Right.

4. Transition model: Describes the new position after each valid move. The robot cannot enter blocked cells

5. Path cost: Each move costs the total cost equals the number of moves.

---

4) Breadth First Search (BFS).

Step	Node	Queue.
Initial	-	[S]
1	S	[A, B]
2	A	[B, C, D]
3	B	[C, D, E]
4	C	[D, E F]
5	D	[E, F]
6	E	[F, G]
7	F	[G]
8	G	[]

Final Path.

S → B → E → G.

Path cost 1 + 1 + 1 = 3

Properties of BFS:

- Time complexity $O(b^{d+1})$
- Space complexity: $O(b^{d+1})$
- Completeness: Yes, for finite searching.

5). Depth First search.

Step	Node	Stack.
Initial	—	[S]
1	S	[B, A]
2	A	[B, D, C]
3	B	[B, D, f]
4	C	[B, D]
5	D	[B]
6	E	[E]
7	F	[G]
8	G	[]

Final Path:

S → B → E → G

Path code 1+1+1 = 3

BFS

Use a queue

Explore level by level

DFS

Use a stack.

Explore deplaye First.

* Time Com.

Finds the Shortest Path for equal edge costs	Does not guratee the shortest path
Usually requires more memory	often requires less memory.

→ BFS is more suitable when the shortest path is required because it guarantees the minimum number of edges in an unweighted graph.